Universidad del Valle de Guatemala

Curso: Programación Orientada a Objetos

Sección #30

Segundo Ciclo, 2026

Catedrática: Erick Marroquin



Lab 4 - Polimorfismo

María del Pilar Ordoñez Helton – 26360

Andrés Martínez Torres – 26633

Propiedades:

#	Tipo de dato	Nombre	Visibilidad	Motivo	Clase
1	Torneo	torneo	private	Referencia el torneo administrado.	Controlador
2	Vista	vista	private	Permite solicitar datos y mostrar información al usuario.	Controlador
3	Scanner	scanner	private	Lee las entradas del usuario desde el teclado.	Vista
4	List<Equipo>	equipos	private	Almacena los equipos registrados.	Torneo
5	Calendario	calendario	private	Administra los eventos y su programación.	Torneo
6	List<Programable>	eventos	private	Almacena partidos y exhibiciones, incluidos los que están sin programar.	Calendario
7	String	codigo	private final	Identifica al equipo; no puede estar vacío ni repetirse en el torneo.	Equipo
8	String	nombre	private final	Guarda el nombre y permite resolver	Equipo

				empates en la tabla.	
9	String	empresa	private final	Guarda la empresa representada.	Equipo
10	int	puntosTabla	private	Acumula los puntos de tabla; inicia en 0.	Equipo
11	Equipo	equipoLocal	private final	Referencia al primer equipo participante.	Partido
12	Equipo	equipoVisitante	private final	Referencia al segundo equipo, distinto del primero.	Partido
13	Integer	marcadorLocal	private	Guarda la anotación local; inicialmente es null.	Partido
14	Integer	marcadorVisitante	private	Guarda la anotación visitante; inicialmente es null.	Partido
15	EstadoPartido	estado	private	Representa los estados sin_programar, programado, finalizado.	Partido
16	LocalDate	fecha	private	Guarda la fecha asignada; inicialmente es null.	Partido

17	Integer	hora	private	Guarda una hora entre 0 y 23; inicialmente es null.	Partido
18	String	sede	private	Guarda la sede asignada; inicialmente es null.	Partido
19	String	titulo	private final	Describe la exhibición.	Exhibicion
20	EstadoExhibicion	estado	private	Representa los estados sin_programar y programada.	Exhibicion
21	LocalDate	fecha	private	Guarda la fecha asignada inicialmente es null.	Exhibicion
22	Integer	hora	private	Guarda una hora entre 0 y 23 inicialmente es null.	Exhibicion
23	String	sede	private	Guarda la sede asignada inicialmente es null.	Exhibicion

Métodos:

#	**Tipo de retorno**	**Nombre**	**Visibilidad**	**Motivo**	**Clase o interfaz**

1	void	main(String[] args)	public static	Crea el controlador e inicia el programa.	Main
2	No aplica	Vista()	public	Constructor: inicializa el Scanner con System.in.	Vista
3	void	mostrarMenu()	public	Muestra las opciones disponibles.	Vista
4	String	leerTexto(String mensaje)	public	Solicita y devuelve un texto.	Vista
5	int	leerEntero(String mensaje)	public	Solicita un entero y repite la lectura si el formato es incorrecto.	Vista
6	LocalDate	leerFecha(String mensaje)	public	Solicita una fecha y repite la lectura si el formato es incorrecto.	Vista
7	void	mostrarMensaje(String mensaje)	public	Muestra confirmaciones y errores.	Vista
8	void	mostrarTabla(List<Equipo> equipos)	public	Muestra los equipos y sus puntos en el orden recibido.	Vista
9	void	mostrarCalendario(Iterable<Programable> eventos)	public	Recorre y muestra los eventos del calendario.	Vista
10	No aplica	Controlador()	public	Constructor: crea el torneo y la vista.	Controlador

11	void	iniciar()	public	Carga los datos iniciales y mantiene activo el menú hasta salir.	Controlador
12	void	cargarDatosIniciales()	private	Crea los equipos, partidos, resultados y exhibiciones iniciales exigidos.	Controlador
13	void	registrarEquipo()	private	Solicita los datos y registra un equipo.	Controlador
14	void	crearPartido()	private	Solicita deporte y equipos, crea el partido y lo agrega al torneo.	Controlador
15	void	crearExhibicion()	private	Solicita el título y agrega la exhibición.	Controlador
16	void	programarEvento()	private	Solicita un evento, fecha, hora y sede para programarlo o reprogramarlo.	Controlador
17	void	registrarResultado()	private	Solicita el partido y su marcador para registrar el resultado.	Controlador
18	No aplica	Torneo()	public	Constructor: inicializa la lista de equipos y crea el calendario.	Torneo

19	void	registrarEquipo(String codigo, String nombre, String empresa)	public	Valida el código y su unicidad, crea el equipo y lo agrega.	Torneo
20	Equipo	buscarEquipo(String codigo)	public	Localiza un equipo por su código.	Torneo
21	void	agregarEvento(Programable evento)	public	Delega el almacenamiento del evento al calendario.	Torneo
22	List<Programable>	obtenerEventos()	public	Obtiene una copia no modificable de todos los eventos.	Torneo
23	void	programarEvento(Programable evento, LocalDate fecha, int hora, String sede)	public	Delega la programación y la validación de conflictos al calendario.	Torneo
24	void	registrarResultado(Puntuable encuentro, int marcadorLocal, int marcadorVisitante)	public	Verifica que el encuentro pertenezca al torneo y solicita registrar el resultado.	Torneo
25	List<Equipo>	obtenerTablaPosiciones()	public	Devuelve una copia de los equipos ordenados por puntos y nombre.	Torneo
26	Iterator<Programable>	iterator()	public	Implementa Iterable delegando el recorrido al calendario.	Torneo

27	No aplica	Calendario()	public	Constructor: inicializa la lista de eventos vacía.	Calendario
28	void	agregarEvento(Programable evento)	public	Almacena el evento y rechaza referencias nulas o el mismo objeto duplicado.	Calendario
29	List<Programable>	obtenerEventos()	public	Devuelve una copia no modificable de todos los eventos.	Calendario
30	void	programarEvento(Programable evento, LocalDate fecha, int hora, String sede)	public	Valida pertenencia, datos y conflictos antes de modificar la programación.	Calendario
31	void	validarConflictos(Programable evento, LocalDate fecha, int hora, String sede)	private	Rechaza conflictos de sede o equipos; excluye al propio evento al reprogramar.	Calendario
32	Iterator<Programable>	iterator()	public	Recorre una copia no modificable de los eventos con fecha, ordenados cronológicamente.	Calendario
33	No aplica	Equipo(String codigo, String nombre, String empresa)	public	Constructor: valida y asigna los datos e	Equipo

				inicia los puntos en 0.	
34	String	getCodigo()	public	Consulta el código.	Equipo
35	String	getNombre()	public	Consulta el nombre.	Equipo
36	String	getEmpresa()	public	Consulta la empresa representada.	Equipo
37	int	getPuntosTabla()	public	Consulta los puntos acumulados.	Equipo
38	void	sumarPuntos(int puntos)	public	Suma puntos de tabla y rechaza cantidades negativas.	Equipo
39	int	compareTo(Equipo otro)	public	Implementa Comparable: ordena por puntos descendentes y luego por nombre alfabético.	Equipo
40	void	programar(LocalDate fecha, int hora, String sede)	public abstract	Define la operación para asignar o cambiar la programación.	Programable
41	LocalDate	getFecha()	public abstract	Define la consulta de la fecha.	Programable
42	Integer	getHora()	public abstract	Define la consulta de la hora.	Programable

43	String	getSede()	public abstract	Define la consulta de la sede.	Programable
44	boolean	estaProgramado()	public abstract	Define la consulta de si el evento tiene programación.	Programable
45	List<Equipo>	getEquiposInvolucrados()	public abstract	Define la consulta de equipos para validar conflictos sin preguntar por el tipo del evento.	Programable
46	String	getDescripcion()	public abstract	Define una descripción del evento para mostrarlo.	Programable
47	int	compareTo(Programable otro)	public default	Implementa una sola vez el orden por fecha, hora y sede para eventos con programación.	Programable
48	void	registrarResultado(int marcadorLocal, int marcadorVisitante)	public abstract	Define el registro de resultados y la distribución de puntos.	Puntuable
49	No aplica	Partido(Equipo equipoLocal, Equipo equipoVisitante)	protected	Constructor: valida equipos distintos y no nulos; inicia el partido sin programar y sin marcador.	Partido
50	void	programar(LocalDate fecha, int hora, String sede)	public	Implementa la programación; valida datos y rechaza	Partido

				cambios si el partido finalizó.	
51	LocalDate	getFecha()	public	Devuelve la fecha asignada.	Partido
52	Integer	getHora()	public	Devuelve la hora asignada.	Partido
53	String	getSede()	public	Devuelve la sede asignada.	Partido
54	boolean	estaProgramado()	public	Indica si tiene programación, incluso después de finalizar.	Partido
55	List<Equipo>	getEquiposInvolucrados()	public	Devuelve una lista no modificable con los dos equipos.	Partido
56	String	getDescripcion()	public	Describe el deporte, equipos, estado y marcador cuando exista.	Partido
57	EstadoPartido	getEstado()	public	Consulta el estado actual.	Partido
58	Integer	getMarcadorLocal()	public	Consulta la anotación local.	Partido
59	Integer	getMarcadorVisitante()	public	Consulta la anotación visitante.	Partido
60	void	registrarResultado(int marcadorLocal, int marcadorVisitante)	public final	Valida estado y marcador, calcula los puntos y después guarda el resultado, suma puntos y finaliza el partido.	Partido

61	void	validarMarcador(int marcadorLocal, int marcadorVisitante)	protected abstract	Define la validación del deporte, adicional al rechazo común de valores negativos.	Partido
62	int	calcularPuntos(int marcadorPropio, int marcadorRival)	protected abstract	Define los puntos de tabla obtenidos según el deporte.	Partido
63	String	getDeporte()	public abstract	Define el nombre del deporte.	Partido
64	No aplica	FutbolSala(Equipo equipoLocal, Equipo equipoVisitante)	public	Constructor: llama a super(equipoLocal, equipoVisitante).	FutbolSala
65	void	validarMarcador(int marcadorLocal, int marcadorVisitante)	protected	Sobrescribe la validación; permite empates, sin restricciones adicionales.	FutbolSala
66	int	calcularPuntos(int marcadorPropio, int marcadorRival)	protected	Sobrescribe el cálculo: 3 por victoria, 1 por empate y 0 por derrota.	FutbolSala
67	String	getDeporte()	public	Sobrescribe el método y devuelve "Fútbol sala".	FutbolSala
68	No aplica	Baloncesto(Equipo equipoLocal, Equipo equipoVisitante)	public	Constructor: llama a super(equipoLocal,	Baloncesto

				equipoVisitante).	
69	void	validarMarcador(int marcadorLocal, int marcadorVisitante)	protected	Sobrescribe la validación y rechaza los empates.	Baloncesto
70	int	calcularPuntos(int marcadorPropio, int marcadorRival)	protected	Sobrescribe el cálculo: 2 por victoria y 1 por derrota para un marcador válido.	Baloncesto
71	String	getDeporte()	public	Sobrescribe el método y devuelve "Baloncesto".	Baloncesto
72	No aplica	Exhibicion(String titulo)	public	Constructor: valida el título e inicia la exhibición sin programar.	Exhibicion
73	void	programar(LocalDate fecha, int hora, String sede)	public	Valida y asigna la programación; establece el estado programada.	Exhibicion
74	LocalDate	getFecha()	public	Devuelve la fecha asignada.	Exhibicion
75	Integer	getHora()	public	Devuelve la hora asignada.	Exhibicion
76	String	getSede()	public	Devuelve la sede asignada.	Exhibicion
77	boolean	estaProgramado()	public	Indica si tiene programación.	Exhibicion
78	List<Equipo>	getEquiposInvolucrados()	public	Devuelve una lista vacía	Exhibicion

				porque no tiene equipos.	
79	String	getDescripcion()	public	Describe el evento usando su título.	Exhibicion
80	String	getTitulo()	public	Consulta el título.	Exhibicion
81	EstadoExhibicion	getEstado()	public	Consulta el estado actual.	Exhibicion

Diseño UML:

